Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

MiniGit: A Lightweight Version Control System Using C++ and Data Structures

Team Name: *Team #*	*Blaze* *DSCPP-III-2026-T296*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Gangwar, Abhinesh – 2510370192* *2510370192@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Jain, Sparsh – 2510370159* *2510370159@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Bajaj, Krish – 2510370469* *2510370469@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Joshi, Snehil – 2510370146* *2510370146@geu.ac.in* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

MiniGit is a lightweight, local, command-line version-control system written in C++. It shows the core ideas of version control in a small form: create a repository, track files, save versions with messages, view history and go back to earlier work, each built on a classic data structure.

Phase 3 closed the restore stage that was behind in Phase 2. `checkout <commit-id>` restores any saved commit without moving a branch, and `rollback <commit-id>` moves the current branch back to an older commit. The team also added branches (`branch`, `checkout <branch>`), a commit graph (DAG) of all branches, and `log --all --graph`. All of this is merged into `main` and passes 13 automated test suites.

The product is one standalone executable with no external library, plus a sample repository with a demo walkthrough, a README and user guide, and a documentation site. All four member modules work together through one command dispatcher.

Updated Project Approach and Architecture

MiniGit keeps the layered design from Phase 2: CLI, command modules, data structures and storage. Every module returns a plain result object (`AddResult`, `CommitResult`, `RestoreResult`, `RollbackResult`, `BranchResult`) and never prints; only the CLI prints, so each module can be tested on its own.

New in Phase 3: branch references in `refs/heads/<name>`; a shared repository layer (`RepoUtils`) that every command uses to read commits and HEAD; and a *detached HEAD*, where HEAD holds a commit id after `checkout <commit-id>`.

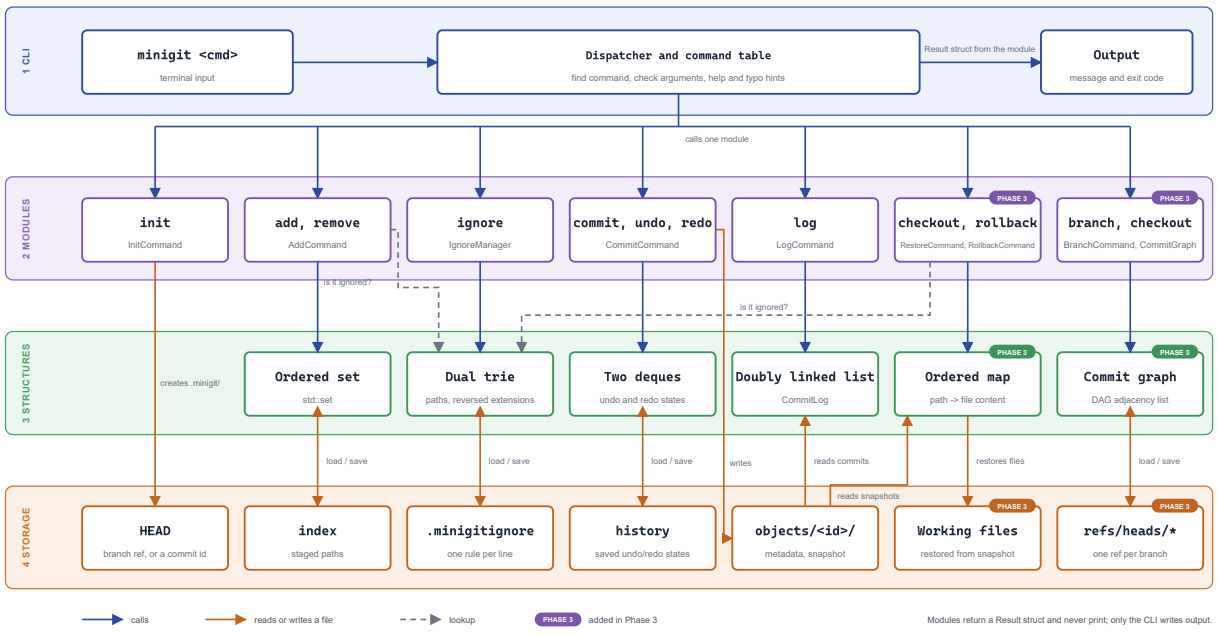
Change from the Phase 2 plan: rollback was planned as a stack of recent restores. It was built instead as a reset to any commit: the branch moves back and the newer commits stay stored. Leaving a checked-out commit is done with `checkout <branch>`.

Data structures:

- **Dual Trie:** forward Trie plus reversed-extension Trie for ignore rules, O(L) per check. Checkout also uses it so ignored files are never written or deleted.
- **Ordered set** (`std::set`): staging index, no duplicates, sorted file.
- **Two deques:** undo/redo states, limited to 20.
- **Doubly linked list:** `log` newest-first or oldest-first in one walk.
- **Commit graph (DAG) as an adjacency list** (`std::unordered_map`): cycle check by three-colour DFS, topological order with a priority queue (newest first), reachability by BFS with a queue. Used by `log --all --graph`.
- **Hash map:** branch name → commit id.
- **Ordered map:** path → file content, used to compare the current and target trees before a restore.

Storage and libraries: `.minigit` holds HEAD, index, history, branch refs and `objects/<id>/` (metadata and a snapshot of the changed files; a commit also lists files it deleted). Commit ids are 40-character SHA-1 hashes from a self-contained implementation. Files are written to a temporary copy and renamed into place so a crash never leaves half a file. Only the C++ standard library is used, with a bundled filesystem library for the C++14 compiler. MiniGit is fully local.

Updated Architecture Diagram



Tasks Completed

Task Completed	Team Member
Integrated the Phase 3 branches into `main`: joined two overlapping branch and checkout designs into one, taught the shared layer a detached HEAD, moved checkout onto the shared commit reader, fixed a Windows-only restore bug. Dispatcher entries and help for `checkout`, `rollback` and `branch`; documentation site with the report viewer and the interactive architecture diagram. Sample repository, demo walkthrough and the live demonstration.	Abhinesh Gangwar
Updated the ignore rules and the Phase 2 modules behind them (Dual Trie, `IgnoreManager`, `StagingArea`, `add`/`remove`/`ignore`).	Sparsh Jain
`rollback <commit-id> [--force]`; `branch`, `branch <name>`, `branch -d`; `checkout <branch> [--force]` with protection for unsaved work; commit graph (DAG) and `log --all --graph`; commits record deleted files; atomic file writes. Rollback, branch, DAG, regression and integration tests.	Krish Bajaj
`checkout <commit-id>` (full id or a unique prefix): refuses first if any unsaved work would be lost, never touches ignored files, writes every file to a temporary copy and swaps it in with a backup so a failure undoes everything, and leaves HEAD detached. Checkout tests, including damaged and unsafe commits.	Snehil Joshi
Phase 2 work (`init`, `add`, `remove`, `ignore`, `commit`, `undo`, `redo`, `log`) is unchanged and its tests still pass.	All members

Challenges/Roadblocks

- **Two versions of the same feature:** Two members built `branch`, `checkout` and the commit graph separately, with the same class names but different designs. The team kept one design for branches and the graph and joined the commit-id checkout into the same `checkout` command. Lesson: one owner per module, agreed before coding.
- **Two readers for one file format:** The checkout code read commit metadata in its own way and did not understand the list of deleted files, so a restore would have brought deleted files back. All commands now read commits through one shared reader, and a test covers deletions.
- **Detached HEAD:** The branch code treated a HEAD holding a commit id as a damaged repository. HEAD may now hold a commit id; `log`, `branch` and `checkout <branch>` work there, while `commit`, `undo`, `redo` and `rollback` refuse and explain how to continue, because they move a branch.
- **Never losing unsaved work:** Checkout and rollback check staged, edited and deleted files first and change nothing if any work would be lost. Checkout writes new files beside the old ones and swaps them in with backups, so a failure part-way restores everything.
- **Keeping ignored files safe:** The Dual Trie is checked before every restore, so ignored files are never written or deleted.
- **Compiler differences on Windows:** On MinGW GCC 6.3 with the bundled filesystem library, a missing file returned a different error code, and a deleted file was reported as unreadable. The check now looks at the file type, which works on every compiler.
- **Unsafe input:** Branch names and stored paths are checked so a name like `../../HEAD` or a path like `../outside.txt` can never reach files outside the repository.

Project Outcome/Deliverables

MiniGit is a working terminal-based version-control application in C++ for local text files. The core operations are `init`, `add`, `remove`, `ignore`, `commit`, `undo`, `redo`, `log`, `checkout` and `rollback`; the branching extensions are `branch`, `branch -d`, `checkout <branch>` and `log --all --graph`.

The submission includes organized C++ source code, 13 automated test suites, a sample repository with a demo walkthrough, a README and `DOCUMENTATION.md` with command examples, a documentation site, this report, and a live demonstration.

The application builds into one standalone executable with no external dependency, and a first-run `install` step lets the user type `minigit` from any terminal.

Progress Overview

All six core operations in the proposal (`init`, `add`, `commit`, `log`, `checkout`, `rollback`) are complete and tested, so the core scope is done. All work is merged into `main` and joined through one command dispatcher.

Ahead of schedule: the Trie-based ignore engine, deque undo/redo, `remove`, first-run installation, branches, the commit graph and `log --all --graph`.

Behind schedule: nothing. The Phase 2 gaps (checkout, rollback, sample repository) are closed.

Future scope: remote repositories, rebase and diffs.

Codebase Information

Repository Link: `https://github.com/strange-exe/MiniGit`
Documentation Site: `https://abhinesh.codes/MiniGit/`
Active Branch: `main` holds the integrated code. Each member works on a personal branch; the team lead merges it into `main` after the full test suite passes.
Layout: `module/include`, `module/src`, `module/tests`, `Reports/`, `sample_repo/`, `README.md`, `DOCUMENTATION.md`, `minigit.exe`

Important Commits:

- Repository setup, CLI parser and table-driven dispatcher, `install` (Abhinesh)
- Trie, Ignore Manager, Staging Area, `add`/`remove`/`ignore`, tests (Sparsh)
- SHA-1 commit ids, `undo`/`redo`; rollback, branches and the commit DAG (Krish)
- `CommitLog` and `log`; `checkout <commit-id>` with a detached HEAD (Snehil)
- Phase 3 merge into `main`: one checkout command, shared commit reader (Abhinesh)

Testing and Validation Status

Test Type	**Status (Pass/Fail)**	**Notes**
Unit test: Trie (test_trie)	Pass	Insert, search, deletion, pruning and path matching.
Unit test: Add and Ignore (test_add)	Pass	Staging, duplicates, ignore rules, staged deletions and the saved index.
Unit test: Init and CLI (test_init, test_cli)	Pass	Repository setup, re-initialisation, argument checks, help, typo hints and the new commands.
Unit test: Commit (test_commit)	Pass	Commit ids, metadata, parent links, undo/redo and the 20-state limit.
Unit test: Log (test_commitlog, test_log)	Pass	The linked list on its own, `log`, `log --reverse`, empty repository.
Unit test: Checkout (test_checkout)	Pass	Exact restore, unsaved-work refusal, ignored files untouched, deletions honoured, detached HEAD rules, damaged or unsafe commits rejected with no change.
Unit test: Rollback (test_rollback)	Pass	Restore of the target commit, later commits kept, protection for unsaved work.
Unit test: Branch and DAG (test_branch, test_dag)	Pass	Create, list, switch, delete; shared ancestors, cycle detection, topological order, graph output.
Regression and integration (test_regression, test_integration)	Pass	326 and 361 checks: whole workflows across commands, path traversal attempts, damaged repositories.
Manual run of the built `minigit.exe`	Pass	`init`, `add`, `commit`, `checkout` by id and by branch, `branch`, refusals while detached, `log --all --graph`.

Deliverables Progress

- **Core CLI pipeline** (`init`, `add`, `commit`, `log`, `checkout`, `rollback`): Completed
- **Dual Trie ignore engine** (`.minigitignore`): Completed
- **Deque undo/redo integration:** Completed
- **Branches and commit graph** (`branch`, `checkout <branch>`, `log --all --graph`): Completed
- **Sample repository and demo walkthrough:** Completed
- **README, DOCUMENTATION.md and documentation site:** Completed
- **Final report and live demonstration:** Report completed; demo ready for evaluation